



Server Setup Guide

Everything you do by hand, in order — and everything the machine does for you. Three accounts, two tokens, one command.

~35 min

of your attention

1

command to run

≈ €9

per month, all in

At a glance

The setup is automated except for the parts that legally need a human with a credit card. Those are the only things on your list.

 **You do this — 5 things**

1. Create the **Hetzner** account

2. Create the **Cloudflare** account and point the domain at it

3. Mint **two API tokens** (copy & paste)

4. Run **one command**

5. Mint **one R2 token**, run the command again

≈ 35 minutes

 **The script does this**

• Creates the tunnel, DNS record and login policy

• Creates the backup bucket

• Generates the database password and encryption keys

• Creates the server, firewalled to SSH only

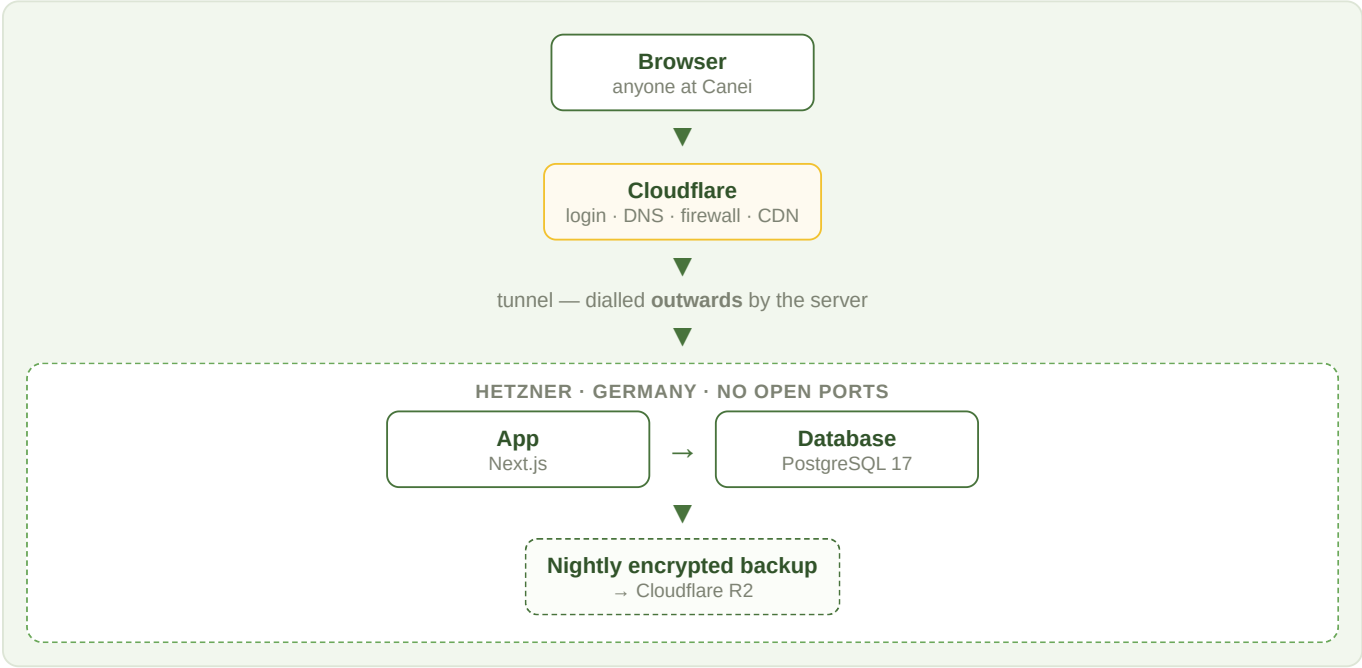
• Installs Docker, the app, the database

• Turns on auto-patching, auto-deploy, nightly backups

• Sets the GitHub secrets

• Waits until the site answers, then tells you

What you end up with



Why no open ports? The server never accepts an incoming connection. It dials out to Cloudflare and traffic comes back down that pipe. There is nothing on the public internet to attack, and no certificate to renew.

WHAT	WHERE	PER MONTH
Server — 4 vCPU, 8 GB, 80 GB	Hetzner, Falkenstein 🇩🇪	€7.05
Whole-server snapshots	Hetzner	€1.40
Encrypted database backups	Cloudflare R2	€0.20
DNS, login, firewall, tunnel	Cloudflare free tier	€0.00
Source, CI, image registry	GitHub free tier	€0.00
Total		≈ €8.65

Step by step

Do these in order. Step 1 first — Hetzner verifies new accounts by hand and that can take a few hours.

Read this before you click anything. Every account goes in **Canei Subirats, S.L.**'s name, with **their** card and a role address like `sistemas@caneisubirats.com` — never a personal mailbox. You add yourself afterwards as an admin and remove yourself at handover. Some providers will not transfer account ownership at all, so doing it the other way round turns the handover into a migration. It costs nothing to get right today.

1

CONSOLE.HETZNER.CLOUD

Create the Hetzner account 10 min

Register with the role address, company name, company address and CIF. Hetzner verifies new company accounts manually — start here so the wait overlaps everything else.

Then create a project called `canei-erp`. That is all — **do not create a server**, the script does that with the right image, firewall and configuration already attached.

2

DASH.CLOUDFLARE.COM

Create the Cloudflare account and add the domain 15 min

Sign up with the same role address. Then add `caneisubirats.com`:

- **Domain already registered elsewhere?** *Add a site*, then change the nameservers at the current registrar to the two Cloudflare gives you. Propagation can take a few hours.
- **Not registered yet?** *Domain Registration* → *Register Domain*. Simplest, because it lands in the right account already.

Then set **SSL/TLS** → **Overview** → **Full (strict)**.

The domain must be **active** in Cloudflare before step 4, or the script stops and tells you so. Nothing is damaged — you just re-run it once the nameservers have propagated.

Step 3 · The two API tokens 10 min

These are what let the script do the other forty things for you. Copy each one straight into the config file — they are shown once.

Hetzner token

Where `console.hetzner.cloud` → project `canei-erp` → **Security** → **API tokens** → Generate
Permission **Read & Write** (Read alone cannot create the server)
Goes in `HCLOUD_TOKEN`

Cloudflare token

Where `dash.cloudflare.com` → **My Profile** → **API Tokens** → Create Token → **Custom token**
Goes in `CF_API_TOKEN`

Add all five permissions:

TYPE	PERMISSION	LEVEL
Account	Cloudflare Tunnel	Edit
Account	Access: Apps and Policies	Edit
Account	Workers R2 Storage	Edit
Zone	DNS	Edit
Zone	Zone	Read

Under **Zone Resources**, include the Canei domain.

You also need your **Account ID** — open any domain in Cloudflare and it is in the right-hand sidebar. That goes in `CF_ACCOUNT_ID`.

Put them in the config file

```
# on your own laptop, in the project folder
cp ops/provision.conf.example ops/provision.conf
nano ops/provision.conf
```

Six values. The file is gitignored, so it never reaches GitHub.

Keep them. Paste both tokens into the customer's password manager as you create them. Cloudflare and Hetzner show each token exactly once.

Step 4 · Run the command 2 min + 10 waiting

```
./ops/provision.sh
```

It checks both tokens before it creates anything, then works through the list. Expect it to pause once:

■ It will stop and ask for one more token

R2 storage tokens cannot be minted reliably through the API, so the script creates the bucket, then stops:

Where `dash.cloudflare.com` → **R2** → **Manage API tokens** → Create API token

Permission **Object Read & Write**

Scope only the bucket `canei-erp-backups` — not "all buckets"

Goes in `R2_ACCESS_KEY_ID` and `R2_SECRET_ACCESS_KEY`

Then run `./ops/provision.sh` again. Everything already done is detected and skipped.

The second run creates the server and waits. The server installs itself while booting — you never log into it. When the script prints **LIVE**, open the URL: Cloudflare asks for an email code, then the ERP loads.

Step 5 · Three things to finish 10 min

A Move the secrets into the password manager

The script wrote everything to `ops/.provisioned/summary.txt`: database password, SSH key, and the backup encryption keys. Copy them into the customer's vault, then delete the folder.

```
cat ops/.provisioned/summary.txt
# ...copy into the password manager, then:
rm -rf ops/.provisioned
```

The one that matters. Lose the `AGE-SECRET-KEY-...` and every backup ever taken becomes permanently unreadable. It is deliberately not stored on the server — that is what stops a compromised server giving away the invoice history.

B Prove a backup restores — today

In GitHub: **Actions** → **Restore drill** → **Run workflow**. It takes the newest backup, decrypts it, restores it into a scratch database and checks it is real.

Green means the company is recoverable. Red means it is not. A backup nobody has restored is a hope, not a backup.

C Narrow SSH to your own address

Hetzner console → **Firewalls** → **canei-erp** → edit the SSH rule, source `<your-ip>/32`. Find your address with `curl ifconfig.me`.

What runs by itself afterwards

WHEN	WHAT HAPPENS
Every 60 seconds	The server checks for a new version and updates itself if CI published one
Nightly, 02:30	Encrypted database backup to Cloudflare R2; anything over 30 days old is removed
Daily	Operating-system security patches install themselves; reboot window 04:00 if a kernel update needs one
1st of the month	Restore drill — a backup is actually restored and checked, and the run fails loudly if it cannot be
Every push to <code>main</code>	CI builds the app, tests it against a real database, and only then publishes it for the server to pick up

Deployments are pull-based. The server fetches new versions; nothing pushes into it. That is why there is no deployment key, no open port, and nothing to steal that would grant access to the box.

If something looks wrong

YOU SEE	IT MEANS	DO THIS
Error 1033	The tunnel is down	<code>docker compose -f docker-compose.prod.yml logs tunnel</code>
502 or 521	The app is down	<code>... logs app --tail 200</code> , then <code>... restart app</code>
Login loops	Access policy mismatch	Zero Trust → Access → check the policy still includes the email domain
Script: <i>domain not in this account</i>	Nameservers have not propagated	Wait, then re-run the script
Script: <i>Hetzner token rejected</i>	Token is Read-only	Regenerate it as Read & Write
Restore drill red	No recoverable backup	Treat as urgent — read the run log, take a manual backup

Full operational detail, including rollback and restore, is in `docs/HANDOVER-OPS.md` — the handbook for whoever maintains this.

When you hand over

The handover is finished when someone else has rebuilt it without you in the room — not when the documents are written.

- ☐ Give the IT provider `docs/HANDOVER-OPS.md` and their own named accounts
- ☐ Run the **rebuild drill** in that document together, onto a throwaway server
- ☐ Transfer the password-manager vault to Canei
- ☐ Remove your address from the Cloudflare Access policy
- ☐ Remove your user from Hetzner, Cloudflare and the GitHub repository
- ☐ Confirm all three accounts bill the customer's card, not yours